Proposal 38: CSP Architect: Browser Security Header Workbench

Assignment: Class 3, Group 6 **Domain:** Web Security **Page:** 1 of 3

Project definition and purpose

Project brief. Design and implement a tool that crawls an allowlisted local web application, evaluates browser security headers, and helps construct a least-privilege CSP. The result must be a functioning, locally demonstrable security system rather than a design-only study. The team must justify its security decisions and show measurable evidence that the implementation works.

Problem and purpose. Security teams often possess raw data but lack a small, understandable system that turns it into a defensible decision. This project must accept controlled input, analyze or protect it using an explicit security method, show an explainable result, and preserve enough evidence for a reviewer to reproduce the decision.

Primary users. Implement an administrator who configures the system and reviews audit records, plus an operator/analyst who runs its normal workflow. Add a restricted viewer if results are exposed to end users. Authorization must be enforced by the server.

Objectives

1. Produce a complete input-to-result workflow for the stated security problem.
2. Make every warning, score, allow/deny decision, or verification result explainable.
3. Protect stored data, credentials, sessions, uploads, and logs according to a threat model.
4. Measure effectiveness with repeatable positive, negative, edge-case, and misuse tests.

Required functional requirements

- **FR-1:** Local crawler. The module must validate its input, report a clear result, and record security-relevant actions without retaining unnecessary sensitive data.
- **FR-2:** Header analysis. The module must validate its input, report a clear result, and record security-relevant actions without retaining unnecessary sensitive data.
- **FR-3:** CSP builder. The module must validate its input, report a clear result, and record security-relevant actions without retaining unnecessary sensitive data.
- **FR-4:** Violation-report collector. The module must validate its input, report a clear result, and record security-relevant actions without retaining unnecessary sensitive data.
- **FR-5:** Before/after score. The module must validate its input, report a clear result, and record security-relevant actions without retaining unnecessary sensitive data.
- **FR-6:** Downloadable configuration. The module must validate its input, report a clear result, and record security-relevant actions without retaining unnecessary sensitive data.

Scope boundary. A responsive web interface, local API, persistent data store, security-analysis/protection component, evidence view, and exportable result are in scope. Production hosting, paid services, native mobile development, real-world attack deployment, and unapproved external targets are out of scope.

CSP Architect: Browser Security Header Workbench — Implementation Specification

Assignment: Class 3, Group 6 **Domain:** Web Security **Page:** 2 of 3

Required architecture

Use a layered design: **(1)** browser interface for validated input and explainable output; **(2)** API/service layer for authentication, authorization, business rules, and safe errors; **(3)** dedicated security engine containing detection, cryptographic, policy, simulation, or forensic logic; **(4)** repository layer for application records and append-only audit events; and **(5)** test-fixture layer containing safe sample inputs. Security logic must not live only in presentation code.

Detailed working sequence

1. The authorized user begins through the **local crawler** module. Unauthorized or invalid input is rejected before processing.
2. The processed result moves to the **header analysis** module. Only validated, normalized data passes to the next stage, together with explainable evidence.
3. The processed result moves to the **csp builder** module. Only validated, normalized data passes to the next stage, together with explainable evidence.
4. The processed result moves to the **violation-report collector** module. Only validated, normalized data passes to the next stage, together with explainable evidence.
5. The processed result moves to the **before/after score** module. Only validated, normalized data passes to the next stage, together with explainable evidence.
6. The system completes the workflow through the **downloadable configuration** module. The final decision, explanation, and supporting evidence are displayed and made available to the approved report/audit record.

Implementation requirements

- Define the data model, roles, input schema, decision/result schema, and audit-event format before building screens.
- Expose small API endpoints for authentication, submission/import, analysis or policy execution, result retrieval, administration, and report export. Use stable error codes and never expose stack traces.
- Build the security engine as independently testable functions/services. Store rule names, model/version details, thresholds, and reasons with each result so decisions can be reproduced.
- Validate file type, size, structure, and fields on the server. Generate identifiers server-side, parameterize database queries, and apply CSRF protection when cookie sessions are used.
- Create a dashboard showing status, severity/confidence where relevant, evidence, recommended action, history, and filters. High-impact actions require authorization and confirmation.
- Log login, configuration change, submission, decision, export, failure, and administrative activity. Redact secrets and unnecessary personal information.

Security and technology

Apply least privilege, secure password hashing or appropriate identity controls, session expiry, server-side authorization, rate limiting on sensitive actions, protected secrets, safe errors, and recovery instructions. Provide a data-flow diagram with trust boundaries and at least eight threats mapped to mitigations.

Suggested stack: Node.js, Express, Playwright optional, HTML parser, SQLite, JavaScript. Equivalent open-source tools may be approved. Everything must run from VS Code on ordinary laptops; Docker is optional.

CSP Architect: Browser Security Header Workbench — Example, Evaluation and Delivery

Assignment: Class 3, Group 6 **Domain:** Web Security **Page:** 3 of 3

Illustrative working example

An authorized operator signs in and supplies a safe, instructor-approved project input. The system validates and normalizes it, runs the configured security mechanism, and returns an explainable decision with evidence. The operator then submits a deliberately invalid or suspicious variant. The system rejects, flags, contains, or distinguishes it as appropriate and records the event. The operator compares both results in history and exports a sanitized report. An administrator changes one permitted rule or threshold, repeats the test, and demonstrates that both the new result and configuration change are traceable.

Expected behavior: A normal sample shows a completed status, relevant evidence, and no unjustified critical warning. A seeded adverse sample identifies the particular indicator or policy violation rather than displaying only “unsafe.” An unauthorized viewer receives a generic denial, and the attempt is logged without exposing the protected record.

Mandatory evaluation cases

- **T1:** Demonstrate XSS test fixtures; record input, expected result, actual result, evidence, and pass/fail status.
- **T2:** Demonstrate inline scripts; record input, expected result, actual result, evidence, and pass/fail status.
- **T3:** Demonstrate third-party-resource simulation; record input, expected result, actual result, evidence, and pass/fail status.
- **T4:** Demonstrate clickjacking; record input, expected result, actual result, evidence, and pass/fail status.
- **T5:** Demonstrate MIME sniffing; record input, expected result, actual result, evidence, and pass/fail status.
- **T6:** Demonstrate allowlist enforcement; record input, expected result, actual result, evidence, and pass/fail status.
- **T-final:** Test malformed/oversized input, an unauthorized request, repeated submission, safe failure behavior, and recovery after restart.

Acceptance criteria

All six functional requirements must be demonstrable. At least 80% of automated tests must pass before final review, with zero unresolved critical authorization, injection, secret-exposure, or unsafe-file-handling defects. Quantitative projects report suitable accuracy/error and performance measures; rule-based projects report coverage, false positives, false negatives, and decision latency. Every report claim must link to a test, screenshot, log excerpt, or measured result.

Delivery, team, and schedule

Deliverables: source and setup guide; sanitized sample data; architecture/data-flow diagrams; threat model; test plan/results; user and administrator guide; limitations and ethics statement; final report; and a reproducible 10–15 minute demonstration. A mock-up alone is insufficient.

Team allocation: Member A—backend/data/authentication; B—security engine/evaluation fixtures; C—frontend/reporting/integration. In a four-person team, D owns automation, security testing, packaging, and QA. Everyone must understand the full system.

Five-week plan: W1—requirements, research, threat model, wireframes, architecture, and test data. W2—data model, authentication, API skeleton, and one end-to-end slice. W3—security engine, dashboard, audit trail, and features. W4—integration, adversarial tests, measurement, accessibility, and hardening. W5—fixes, evaluation, documentation, packaging, and demo rehearsal.